Algorithmen und Datenstrukturen
Übung 7 – Sortierende Container

Aufgabe 1: Theorie

a) Führen Sie die folgenden Operationen auf einem sortierenden Binärbaum aus. Der Baum soll keine Selbst-Balancierung verwenden. Zeichnen Sie den Baum nach jeder Veränderung (z. B. nach jedem Teilschritt beim Löschen).

- Fügen Sie die nachstehenden Werte **in exakt der gegebenen Reihenfolge** in den Baum ein.

  ```
  16, 4, 9, 32, 1, 2, 3, 24, 33, 35, 37
  ```

- Welchen der folgenden Werte enthält der Baum? Geben Sie an, welche Knoten jeweils geprüft werden, bis der Wert gefunden oder definitiv nicht gefunden ist.

  ```
  16, 37, 35, 39, 3, 12
  ```

- Löschen Sie nachstehende Werte **in exakt der gegebenen Reihenfolge** aus dem Baum.

  ```
  35, 1, 16, 37
  ```

b) Fügen Sie folgende Werte in einen Baum in exakt der gegebenen Reihenfolge ein.

```
8, 4, 12, 2, 6, 10, 14
```

Im Baum sollen folgende Werte gefunden werden:

```
8, 12, 6
```

Wie viele Schritte braucht der Suchalgorithmus jeweils?

c) Fügen Sie folgende Werte in einen Baum in exakt der gegebenen Reihenfolge ein.

```
2, 4, 6, 8, 10, 12, 14
```

Im Baum sollen folgende Werte gefunden werden:

```
8, 12, 6
```

Wie viele Schritte braucht der Suchalgorithmus jeweils? Wo besteht ein Zusammenhang zu dem Baum der vorherigen Teilaufgabe?

d) Fügen Sie die folgenden Werte in einen AVL-Baum ein. Notieren Sie zu jedem Knoten dessen Balance-Wert. Im Fall von Rotationen zeichnen Sie den Baum **vor** sowie **nach** der Rotation und geben Sie an, was für eine Rotation jeweils nötig ist.

```
2, 4, 6, 8, 10, 12, 20, 16, 17, 18, 19
```

Aufgabe 2: Praxis

a) Programmieren Sie zwei Klassen `tree` und `node` in C++. Die Klassen sollen einen binären Suchbaum modellieren und nachstehende Funktionen erhalten:

- Der Suchbaum verwaltet Einträge vom Typ `int` in den Knoten.

  Alternativ für Profis: Der Suchbaum verwaltet Einträge von einem beliebig (aber nur für einen ganzen Baum) wählbaren Typ `T`.

- Der `tree` speichert eine `size` für die Anzahl an Einträgen im Baum sowie einen Verweis auf einen (optionalen) Wurzelknoten `root`.
- Die `node` speichert Verweise auf (optionale) Kinder `left` und `right` und auf einen (optionalen) Elternknoten `parent`.
- Konstruktor eines leeren `tree`s (d. h. `size` ist 0 und keine `root`).
- Destruktor, der einen `tree` korrekt mit allen enthaltenen `node`s löscht.
- Es gibt eine Funktion `insert`, welche einen gegebenen Eintrag an der richtigen Stelle im Suchbaum einfügt (sofern der Eintrag noch nicht drin ist).
- Es gibt eine Funktion `erase`, die einen gegebenen Eintrag aus dem Baum entfernt (sofern der Eintrag im Baum enthalten ist).
- Es gibt eine Funktion `contains`, welche prüft, ob ein gegebener Eintrag in dem Baum enthalten ist.
- Es gibt eine Funktion `print_width_first`, die alle Einträge im Baum schichtweise auf der Kommandozeile ausgibt.
- Es gibt eine Funktion `validate`, welche prüft, ob der Baum die Suchbaumeigenschaft erfüllt und ob die Verzeigerung passt (d. h. ob die Zahl der Einträge im Baum der `size` entspricht und ob die `parent`-Zeiger von Kindknoten zu den `left`- bzw. `right`-Zeigern der Elternknoten passen).

b) Schreiben Sie eine `main`-Funktion, welche den Baum prüft:

- Wählen Sie eine zufällige Zahl zwischen 0 und 10000.
- Fügen Sie so viele zufällig gewählte Einträge in den Baum ein, wie die vorher bestimmte Zufallszahl angibt.
- Prüfen Sie mit validate(), ob der Baum in Ordnung ist, und prüfen Sie, ob jeder eingefügte Eintrag tatsächlich enthalten ist.
- Wiederholen Sie den obigen Ablauf mehrere dutzend Mal (jeweils mit neuen Zufallszahlen). Dies ist sogenanntes *fuzzy testing* (oder kurz: *fuzzying*), das Testen mit randomisierten Daten.

c) Ändern Sie den Baum der vorherigen Aufgaben wie folgt zu einem AVL-Baum ab:

- **Heben Sie die alte Implementierung zu Vergleichszwecken auf!**
- Die `node` speichert einen Balance-Wert `balance`.

- Der Balance-Wert wird beim Einfügen angepasst. Sollte der Balance-Wert den zulässigen Wertebereich überschreiten, wird der Baum passend durch Rotationen angepasst.
- Die `validate`-Funktion prüft zusätzlich, ob die `balance` in Ordnung ist.

d) Testen Sie den AVL-Baum erneut mit der `main` der vorherigen Teilaufgabe.

e) Erstellen Sie einen zufälligen, extrem großen Baum (z. B. eine Million zufällig gewählte Einträge). Messen Sie jeweils für die AVL-Baum-Implementierung und für die gewöhnliche Implementierung, wie lange das Einfügen bzw. das Finden eines Eintrags dauert. Können Sie die Erkenntnisse der Vorlesung bestätigen?